

We thank the reviewers for their helpful and detailed comments. We address the major comments and we will incorporate all the comments in our revision. Additionally, demos of WebRelate can be found at <https://drive.google.com/open?id=0ByGcuXqa9Z3KVEhERjBmSXJRc0U>.

Common Questions

Q. Delta over FlashFill is incremental? Contributions to PL?

This paper presents a novel program synthesis paradigm of “Output-constrained Programming By Examples”. This formulation is only possible in situations where it is possible to compute a finite set of possible outputs for new inputs (i.e. inputs other than the inputs in the I/O examples). FlashFill (and its extensions) do not have such a property as any output string is equally likely for the new inputs and there is no way to constrain the possible set of outputs. The output-constrained PBE allows us to come up with a new synthesis algorithm that combines both the output uniqueness constraint (program should match only 1 output string for user-provided examples) as well as the generalization constraint (output is within the set of possible outputs for other inputs). We instantiate this paradigm for two domains needed for data-dependent web extraction: i) URL generation, ii) data-dependent web data extraction. We believe this new synthesis paradigm will open up a new class of synthesis techniques and applications in future.

Q. Only 3 examples without output-constrained ranking?

Note that Fig 25 compares output-constrained ranking with the layered VSA based ranking. The layered VSA already has a strong prior that the programs in earlier layers are more likely to be the desired one as opposed to programs in later layers, so it is quite a strong baseline comparison. The FlashFill based learning requires more than 5 examples for smaller length URL strings and the synthesis algorithm doesn't even scale for longer URLs. We can add those comparison graphs as well if reviewers would like.

Reviewer A

Q. Novelty of the Xpath synthesis algorithm?

Unlike previous Xpath learning algorithms, our synthesis algorithm for data extraction uses VSA based synthesis that allows us to seamlessly integrate with complex string transformations. The ability to use string transformations is essential for data-dependent extractions. Additionally, this is another domain where output-constrained PBE is possible, and our algorithm exploits that to efficiently prune away programs that do not generalize.

Q. Unfair criticism of record-replay based approaches?

We didn't mean to harshly criticize these approaches; we only wanted to convey the fact that there is an alternate algorithmic way to reduce the interactions needed from users for at least a class of web data integration tasks. But there are definitely tasks beyond the ones WebRelate can support such as traversing multiple webpages or requiring complex UI interactions that requires more sophisticated user demonstrations. We will restate this more clearly and also clarify the scenarios where record-replay based approaches are essential and complementary to WebRelate.

Reviewer B

Q. Fig 22 is confusing: what are the categories and why use L4?

Yes, you are correct. We categorize the URL learning benchmarks based on the layer that contains the desired program. Yes, none of the programs need the layer 4 in our benchmarks. We still use layer 4 in our experiments for two reasons: i) for completeness of the synthesis algorithm for our DSL for other benchmarks in future, and ii) for comparing against a no layered approach (a baseline similar to FlashFill like VSA algorithms without a layered approach).

Q. Why show the graph complexity with # of examples?

We wanted to show the practicality of the synthesis algorithm due to the layered Version-space and the lazy intersection algorithms. Even though the theoretical complexity of the synthesis algorithms is exponential in the number of examples in worst case, we observe surprisingly that they scale linearly in practice.

Q. Formal Theorems/Proofs?

We will make the proofs more formal by formally defining consistency, soundness, and completeness of the algorithms.

Reviewer C

Q. Are there other domains where learning from noisy oracles will fit?

We presented two such domains: URL learning and input-dependent data extraction from webpages. There are definitely many scenarios where such a paradigm might be useful. For example, many tasks where labels are available from (noisy) mechanical turks, such as image captioning or natural language commands, our technique could be applicable to learn a program such that the result on new inputs is amongst the set of labels for that input.

Q. Have other systems used layered VSA? Can it be useful for other systems?

Not to the best of our knowledge. Yes, all previous VSA systems can potentially benefit from the layered VSA approach given that the layers are designed with the right insights.

Q. How were the examples selected?

For some benchmarks, we had scenarios with examples and webpages provided by the Excel product team. For other benchmarks, we chose alternate sources for data extraction and provided examples manually, but they were independent of the underlying learning system. We used a part of the initial Excel examples to design the DSL and ranking. We will elaborate more on this process of creating the benchmark set and selecting the examples in the revision.

Q. Two-phase approach with WebCombine can perform data-dependent extraction?

No, first of all systems like WebCombine do not allow for sophisticated regular expression based string-extraction tasks. Second, sometimes the data-dependence between input and webpages is hidden, e.g. a stock div element might have id “msft-price”, which won’t be directly visible to the users, so they might not be able to identify and provide examples regarding what data should be extracted into the structured format in the intermediate step before the second step of data extraction.